

Semana 1 — Introducción a Python para IA

Ejercicios de clase — Versión Estudiante

Hilo conductor: todos los ejercicios de esta semana giran en torno al mismo escenario: estás construyendo, en Python puro (sin librerías todavía), las piezas básicas de un pequeño sistema para registrar experimentos de entrenamiento de modelos de Machine Learning — nombre del modelo, accuracy obtenida, número de épocas, tamaño del dataset, si convergió o no. La idea es dominar la sintaxis del lenguaje antes de entrar a NumPy/Pandas en las próximas semanas.

Cada ejercicio tiene: contexto, enunciado, código de partida (completa donde veas `# TODO` o `_____`) y el resultado esperado en consola.

Ejercicio 1 — Explorando el intérprete interactivo de Python

Aprendizaje específico: Experimenta con el intérprete de Python.

Contexto: Antes de escribir scripts, en IA se pasa mucho tiempo en el intérprete interactivo (REPL) probando ideas rápido — es la forma más directa de probar una hipótesis sobre datos antes de escribirla en un notebook o script.

Enunciado: Abre una terminal y ejecuta `python3` (o `python`) para entrar al intérprete. Ejecuta, una por una, y anota qué imprime cada línea:

```
>>> 3 + 4 * 2
>>> type(3.5)
>>> "IA" * 3
>>> nombre_modelo = "RandomForest"
>>> nombre_modelo
>>> dir(nombre_modelo)[-5:]
>>> help(str.upper)
>>> exit()
```

Responde por escrito: ¿qué diferencia notas entre escribir código en el intérprete y ejecutar un archivo `.py`? ¿Para qué sirve `dir()`? ¿Qué te mostró `help()`?

Resultado esperado: no hay un único "resultado" — se evalúa que hayas ejecutado cada línea y puedas explicar qué hace cada una.

Ejercicio 2 — Variables y tipos de datos

Aprendizaje específico: Aplica lenguaje de programación Python.

Contexto: Vas a registrar los datos de un experimento de entrenamiento.

Enunciado: Declara variables para representar un experimento, usando el tipo de dato correcto para cada uno:

- Nombre del modelo: "RandomForest"
- Accuracy obtenida: 0.87
- Número de épocas: 20
- ¿Convergió?: True
- Observaciones: sin observaciones (usa el valor que representa "nada" en Python)

Imprime cada variable junto con su tipo usando `type()`.

Código de partida:

```
nombre_modelo = ____
accuracy = ____
epocas = ____
convergio = ____
observaciones = ____

print(nombre_modelo, type(nombre_modelo))
print(accuracy, type(accuracy))
print(epocas, type(epocas))
print(convergio, type(convergio))
print(observaciones, type(observaciones))
```

Resultado esperado:

```
RandomForest <class 'str'>
0.87 <class 'float'>
20 <class 'int'>
True <class 'bool'>
None <class 'NoneType'>
```

Ejercicio 3 — Operadores

Aprendizaje específico: Aplica lenguaje de programación Python.

Contexto: Quieres comparar dos experimentos y calcular métricas derivadas.

Enunciado: Dadas las accuracy de dos experimentos, calcula el error de cada uno (`1 - accuracy`), la diferencia de accuracy entre ambos, y determina con un operador de comparación cuál modelo es mejor.

Código de partida:

```
accuracy_modelo_a = 0.87
accuracy_modelo_b = 0.91
```

```
error_a = ____
error_b = ____
diferencia = ____
b_es_mejor = ____ # True si b tiene mayor accuracy que a

print(f"Error A: {error_a:.2f}")
print(f"Error B: {error_b:.2f}")
print(f"Diferencia de accuracy: {diferencia:.2f}")
print(f"¿B es mejor que A?: {b_es_mejor}")
```

Resultado esperado:

```
Error A: 0.13
Error B: 0.09
Diferencia de accuracy: 0.04
¿B es mejor que A?: True
```

Ejercicio 4 — Listas

Aprendizaje específico: Aplica lenguaje de programación Python.

Contexto: Corriste 5 experimentos y quieres manejar sus accuracy como una colección.

Enunciado: Crea una lista con las accuracy [0.75, 0.82, 0.91, 0.68, 0.87]. Agrega un sexto valor 0.95, elimina el valor 0.68, ordénala de mayor a menor, e imprime el mejor resultado (primer elemento tras ordenar) y el total de experimentos.

Código de partida:

```
accuracies = [0.75, 0.82, 0.91, 0.68, 0.87]

# TODO: agregar 0.95 al final

# TODO: eliminar 0.68

# TODO: ordenar de mayor a menor

print(accuracies)
print("Mejor accuracy:", ____)
print("Total de experimentos:", ____)
```

Resultado esperado:

```
[0.95, 0.91, 0.87, 0.82, 0.75]  
Mejor accuracy: 0.95  
Total de experimentos: 5
```

Ejercicio 5 — Tuplas y conjuntos

Aprendizaje específico: Aplica lenguaje de programación Python.

Contexto: Los hiperparámetros de un experimento ya cerrado no deberían cambiar (tupla). Además, quieres saber cuántos modelos *distintos* has probado en total (conjunto, sin duplicados).

Enunciado:

1. Crea una tupla `hiperparametros` con `(0.01, 20, "adam")` (learning rate, épocas, optimizador). Intenta modificar el primer valor y observa (comenta en el código) qué error lanza Python.
2. Crea un conjunto `modelos_probados` a partir de esta lista con nombres repetidos: `["RandomForest", "SVM", "RandomForest", "KNN", "SVM"]`. Imprime el conjunto resultante y su longitud.

Código de partida:

```
hiperparametros = ____  
# TODO: descomenta la siguiente línea, ejecútala para ver el error, y vuelve a  
comentarla  
# hiperparametros[0] = 0.1  
  
nombres_con_repetidos = ["RandomForest", "SVM", "RandomForest", "KNN", "SVM"]  
modelos_probados = ____  
  
print(hiperparametros)  
print(modelos_probados)  
print("Modelos distintos probados:", ____)
```

Resultado esperado (el orden del set puede variar):

```
(0.01, 20, 'adam')  
{'RandomForest', 'SVM', 'KNN'}  
Modelos distintos probados: 3
```

Y como comentario en tu código, el error que obtuviste al intentar modificar la tupla.

Ejercicio 6 — Diccionarios

Aprendizaje específico: Aplica lenguaje de programación Python.

Contexto: Un experimento tiene varios campos relacionados entre sí — un diccionario representa esto mejor que variables sueltas.

Enunciado: Representa el experimento del Ejercicio 2 como un diccionario con llaves "modelo", "accuracy", "epocas", "convergio". Agrega una nueva llave "dataset_size" con valor 5000. Imprime el valor de accuracy usando la llave, y luego todas las llaves del diccionario.

Código de partida:

```
experimento = {  
    ____  
}  
  
# TODO: agregar la llave "dataset_size" con valor 5000  
  
print("Accuracy:", ____)  
print("Llaves:", ____)
```

Resultado esperado:

```
Accuracy: 0.87  
Llaves: dict_keys(['modelo', 'accuracy', 'epocas', 'convergio', 'dataset_size'])
```

Ejercicio 7 — Condicionales

Aprendizaje específico: Aplica lenguaje de programación Python.

Contexto: Necesitas clasificar automáticamente si un experimento es aceptable para producción.

Enunciado: Para cada valor de accuracy en la lista dada, imprime:

- "Excelente" si accuracy ≥ 0.9
- "Aceptable" si $0.75 \leq \text{accuracy} < 0.9$
- "Insuficiente" en cualquier otro caso

Adicionalmente, usa match para imprimir el nombre completo de un optimizador a partir de su abreviación ("adam", "sgd", "rms"; cualquier otro caso imprime "Desconocido").

Código de partida:

```
for accuracy in [0.95, 0.80, 0.60]:  
    if ____:  
        print("Excelente")  
    elif ____:  
        print("Aceptable")  
    else:
```

```
print(____)

optimizador = "sgd"
match optimizador:
    case "adam":
        print("Adaptive Moment Estimation")
    case ____:
        print(____)
    case ____:
        print(____)
    case _:
        print("Desconocido")
```

Resultado esperado:

```
Excelente
Aceptable
Insuficiente
Stochastic Gradient Descent
```

Ejercicio 8 — Bucles

Aprendizaje específico: Aplica lenguaje de programación Python.

Contexto: Tienes una lista de experimentos (diccionarios) y necesitas recorrerla para sacar estadísticas.

Enunciado: Usa un **for** para calcular la suma total de accuracy y encontrar el modelo con mejor accuracy. Luego usa un **while** para imprimir una cuenta regresiva de "épocas restantes" desde 3 hasta 0.

Código de partida:

```
experimentos = [
    {"modelo": "RandomForest", "accuracy": 0.87},
    {"modelo": "SVM", "accuracy": 0.91},
    {"modelo": "KNN", "accuracy": 0.78},
]

suma_accuracy = 0
mejor_modelo = None
mejor_accuracy = 0

for exp in experimentos:
    ____ # sumar exp["accuracy"] a suma_accuracy
    if ____:
        mejor_accuracy = exp["accuracy"]
        mejor_modelo = exp["modelo"]

promedio = suma_accuracy / len(experimentos)
```

```
print(f"Promedio de accuracy: {promedio:.3f}")
print(f"Mejor modelo: {mejor_modelo} ({mejor_accuracy})")

epocas_restantes = 3
while ____:
    print("Épocas restantes:", epocas_restantes)
    ____
```

Resultado esperado:

```
Promedio de accuracy: 0.853
Mejor modelo: SVM (0.91)
Épocas restantes: 3
Épocas restantes: 2
Épocas restantes: 1
```

Ejercicio 9 — Funciones

Aprendizaje específico: Aplica lenguaje de programación Python.

Contexto: La lógica de clasificar accuracy (Ejercicio 7) y calcular promedios (Ejercicio 8) se repite — momento de encapsularla en funciones reutilizables.

Enunciado: Escribe `clasificar_experimento(accuracy)` que **retorne** (no imprima) "Excelente", "Aceptable" o "Insuficiente". Escribe `promedio_accuracy(lista_experimentos)` que retorne el promedio de accuracy de una lista de diccionarios. Usa una función `lambda` para ordenar la lista de experimentos por accuracy, de mayor a menor.

Código de partida:

```
def clasificar_experimento(accuracy):
    ____

def promedio_accuracy(lista_experimentos):
    ____

experimentos = [
    {"modelo": "RandomForest", "accuracy": 0.87},
    {"modelo": "SVM", "accuracy": 0.91},
    {"modelo": "KNN", "accuracy": 0.78},
]

print(clasificar_experimento(0.91))
print(f"Promedio: {promedio_accuracy(experimentos):.3f}")

experimentos_ordenados = sorted(experimentos, key=____)
print([e["modelo"] for e in experimentos_ordenados])
```

Resultado esperado:

```
Excelente
Promedio: 0.853
['SVM', 'RandomForest', 'KNN']
```

Ejercicio 10 — Manejo de errores

Aprendizaje específico: Aplica lenguaje de programación Python.

Contexto: Los datos de un experimento a veces vienen mal capturados (accuracy como texto, o fuera de rango). El código no debería romperse por eso.

Enunciado: Escribe `validar_accuracy(valor)` que intente convertir `valor` a `float` y verifique que esté entre 0 y 1. Debe manejar con `try/except` el caso en que `valor` no se pueda convertir a número, y lanzar (`raise`) un error propio cuando esté fuera de rango. Pruébala con `"0.87"`, `"abc"` y `"1.5"`.

Código de partida:

```
def validar_accuracy(valor):
    try:
        numero = ____
        if ____:
            raise ValueError("Accuracy fuera de rango")
        return numero
    except ValueError as e:
        print(f"Dato inválido ({valor}): {e}")
        return None

for valor in ["0.87", "abc", "1.5"]:
    resultado = validar_accuracy(valor)
    print("Resultado:", resultado)
```

Resultado esperado:

```
Resultado: 0.87
Dato inválido (abc): could not convert string to float: 'abc'
Resultado: None
Dato inválido (1.5): Accuracy fuera de rango
Resultado: None
```

Ejercicio 11 — Manejo de archivos

Aprendizaje específico: Aplica lenguaje de programación Python.

Contexto: No quieres perder el historial de experimentos cuando cierras el programa — hay que guardarlo en disco.

Enunciado: Usando `with open(...)`, escribe en `historial.txt` una línea por experimento con formato `modelo,accuracy`. Reabre el archivo en modo lectura y, por cada línea, imprime `modelo` y `accuracy` separados con `.split(",")`.

Código de partida:

```
experimentos = [  
    {"modelo": "RandomForest", "accuracy": 0.87},  
    {"modelo": "SVM", "accuracy": 0.91},  
]  
  
with open("historial.txt", "w") as archivo:  
    for exp in experimentos:  
        archivo.write(f"{exp['modelo'], exp['accuracy']}")  
  
with open("historial.txt", "r") as archivo:  
    for linea in archivo:  
        modelo, accuracy = linea.split(",")  
        print(f"Modelo: {modelo}, Accuracy: {accuracy.strip()}")
```

Resultado esperado:

```
Modelo: RandomForest, Accuracy: 0.87  
Modelo: SVM, Accuracy: 0.91
```

Ejercicio 12 — Módulos

Aprendizaje específico: Aplica lenguaje de programación Python.

Contexto: Todo lo construido en los ejercicios anteriores está en un solo archivo. Un proyecto real separa esto en módulos: uno con la lógica, otro con el punto de entrada.

Enunciado: Crea dos archivos:

- `experimentos.py`: contiene `clasificar_experimento` y `promedio_accuracy` del Ejercicio 9.
- `main.py`: importa esas funciones desde `experimentos.py` y las usa sobre una lista de experimentos para imprimir el promedio y la clasificación del mejor experimento.

Código de partida (`experimentos.py`):

```
def clasificar_experimento(accuracy):  
    _____
```

```
def promedio_accuracy(lista_experimentos):  
    _____
```

Código de partida (main.py):

```
from ____ import ____  
  
experimentos = [  
    {"modelo": "RandomForest", "accuracy": 0.87},  
    {"modelo": "SVM", "accuracy": 0.91},  
    {"modelo": "KNN", "accuracy": 0.78},  
]  
  
promedio = ____  
mejor = max(experimentos, key=lambda e: e["accuracy"])  
  
print(f"Promedio: {promedio:.3f}")  
print(f"Mejor modelo ({mejor['modelo']}):  
{clasificar_experimento(mejor['accuracy'])}")
```

Resultado esperado (al ejecutar main.py):

```
Promedio: 0.853  
Mejor modelo (SVM): Excelente
```